

INFORME TÉCNICO: PRÁCTICA 4 – PROCESAMIENTO E INTEROPERABILIDAD DE DATOS DE SENSORES TÁCTILES

Asignatura: Sistemas Operativos

Titulación: Grado en Tecnologías Digitales para la Empresa

Alumno: Nidhal Majoul

Repositorio GitHub: <https://github.com/nidhal-majoul/PRACTICA4>

1. Explicación del sistema

El sistema gestiona de forma secuencial un flujo de datos provenientes de un sensor táctil industrial que traduce la fuerza mecánica en valores numéricos discretos. El software se compone de dos subsistemas distribuidos e independientes conectados en red local:

- **Cliente (C++):** Lee secuencialmente el archivo JSON `tactile_captures_50.json` (50 capturas históricas), valida la integridad estructural de cada matriz y ejecuta de manera manual un algoritmo de procesamiento geométrico para elevar su resolución espacial. Finalmente, empaqueta los datos en formato JSON y los transfiere mediante peticiones HTTP POST.
- **Servidor (Python):** Un servicio web basado en `Flask` que escucha en el puerto 5000. Recibe el payload, decodifica la matriz de presión mediante `NumPy` y genera mapas cromáticos estandarizados en formato `.png` para su almacenamiento y análisis operativo.

2. Explicación de la interpolación

La lectura física del hardware ofrece una rejilla reducida de 16×16 posiciones (256 puntos). Para su correcta monitorización visual, el cliente "estira" la matriz hasta una resolución de 128×128 elementos (16.384 puntos) mediante un algoritmo manual de **interpolación bilineal**.

El mapeo calcula un factor de escala flotante continuo mediante la ecuación $\text{escala} = (16-1)/(128-1) \approx 0.11811$. Para cada celda destino (i,j), se determina su posición equivalente original: $\text{fila} = i \times \text{escala}$ y $\text{columna} = j \times \text{escala}$.

Utilizando las funciones `[columna]`, `[fila]`, `[columna]` y `[fila]` (con control de límites para no superar el índice físico 15), el programa localiza los 4 píxeles reales vecinos más cercanos (p_1, p_2, p_3, p_4) y sus distancias residuales (dx,dy). El valor entero de presión final se calcula aplicando la ponderación espacial: $\text{Valor} = p_1(1-dx)(1-dy) + p_2 \cdot dx(1-dy) + p_3(1-dx)dy + p_4 \cdot dx \cdot dy$.

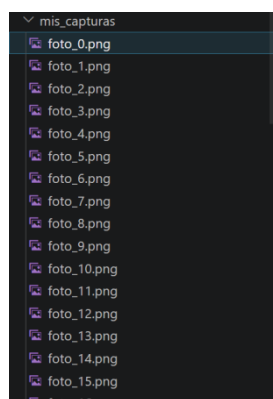
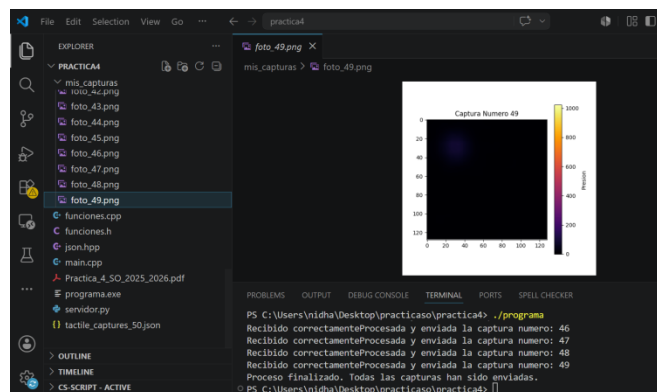
3. Arquitectura cliente-servidor

El sistema implementa una arquitectura Cliente-Servidor acoplada sobre el protocolo de aplicación HTTP.

- **Modularidad en C++:** El diseño divide las responsabilidades en archivos independientes (`funciones.h` para prototipos, `funciones.cpp` para lógica matemática y `main.cpp` para el control de flujo del bucle).
- **Interoperabilidad por Archivo Temporal:** Para evitar dependencias complejas de librerías dinámicas de red en la compilación sobre Windows, C++ vuelca la matriz serializada en un archivo local imperceptible llamado `temp.json`. Acto seguido, invoca al binario `curl` nativo del sistema operativo mediante la llamada del sistema `system()` con la instrucción: `curl -s -X POST -H "Content-Type: application/json" -d @temp.json http://127.0.0.1:5000/procesar`. Tras la confirmación del socket, el proceso limpia el directorio llamando a `remove("temp.json")`.
- **Servidor Flask:** El backend de Python permanece en escucha pasiva en la dirección local `127.0.0.1:5000` procesando las entradas de red.

4. Capturas de resultados

Ambos subsistemas interactúan en tiempo real de manera síncrona. El cliente C++ procesa el bucle completo y el servidor Flask genera secuencialmente las 50 imágenes en la carpeta `mis_capturas/`.



5. Problemas encontrados

1. **Saturación de Comillas en Consola (C++):** Pasar la matriz JSON como texto plano directamente en el argumento de `system()` provocaba fallos de sintaxis en la consola de Windows (`cmd/PowerShell`). Se solventó abstrayendo el payload mediante el archivo de intercambio intermedio (`-d @temp.json`).
2. **Colapso por Hilos de Interfaz Gráfica (Python):** El servidor caía tras recibir pocas peticiones con el error `RuntimeError: main thread is not in main loop` debido a que `matplotlib` intentaba instanciar ventanas físicas con `tkinter` por cada petición web. Se solucionó forzando al backend a operar en modo *Headless* (renderizado directo en memoria de intercambio) usando la directiva `matplotlib.use('Agg')` antes de las llamadas gráficas.

6. Conclusiones técnicas

1. **Ventajas de JSON sobre Sockets Puros:** La serialización estructurada en JSON elimina los problemas clásicos de alineación de bytes (*Endianness*) e incompatibilidad de tipos entre lenguajes heterogéneos de bajo y alto nivel (C++ y Python).
2. **Eficiencia en Sistemas Distribuidos:** Ejecutar la carga matemática pesada (interpolación bilineal) en el nodo periférico cliente optimiza el ancho de banda y mitiga la saturación del servidor centralizado.
3. **Estabilidad en Servicios de Red:** En aplicaciones de servidor distribuidas es obligatorio desacoplar las interfaces gráficas interactivas (GUI) en favor de subprocesos en memoria de intercambio para garantizar la disponibilidad del servicio ante ráfagas masivas de datos.